"Humain ou IA ?" dans Minecraft : note d'avancement

SAE S5 DATAIA, sujet 5. Groupe : Matteo Godart, Ayoub Seghir, Yacine Mounsi. 22 octobre 2026.

1. Le projet en bref

Notre sujet consiste à faire jouer ensemble des humains et des bots sur un serveur Minecraft. Les bots sont pilotés par un modèle de langage (LLM). Régulièrement, chaque joueur doit dire qui, selon lui, est un humain et qui est une IA. On cherche à savoir si nos bots passent pour des humains, et sur quoi on les reconnaît : le temps qu'ils mettent à répondre, leur façon d'écrire, la logique de leurs actions.

Le serveur (PaperMC 1.20.4) tourne sur notre VM de l'université. Les trois bots, Joueur4, Joueur5 et Joueur6, se connectent comme de vrais joueurs grâce à la bibliothèque Mineflayer. Elle est écrite en Node.js, ce qui explique ce choix de langage (autorisé pour le sujet 5). Les bots n'ont aucun droit d'administration : ils voient et font seulement ce qu'un joueur peut faire. Ils appellent le modèle `gpt-oss-120b` (un modèle Mistral derrière ce nom) uniquement par la passerelle LiteLLM de l'université. Chaque partie est enregistrée dans des journaux (appels au modèle, décisions, actions, messages, réflexions), que l'on analyse avec des scripts Python.

Pour les gestes du jeu (miner, fabriquer, cuire, poser des blocs), on a repris la bibliothèque du projet Mindcraft, cité dans le sujet. Nos premières actions faites maison échouaient trop souvent : 17 sur 37 lors d'un test. Voyager, l'autre projet cité, a été écarté : le modèle y écrit son propre code, ce qui ne fonctionne bien qu'avec GPT-4.

2. Où on en est, et les étapes clés

Partie	État
VM, serveur Minecraft, accès à la passerelle	fait
Bots : 34 actions, survie, combat avec des temps de réaction humains	fait, à améliorer
Chat : délais, fautes de frappe, personnalités (étudiants majeurs de filières différentes)	fait, à améliorer
Réflexion toutes les 3 minutes (résumé, avis sur les autres, objectif)	fait
Coopération entre bots, construction de structures	en cours
Journaux, scripts d'analyse, banc d'essai du chat, 69 tests automatiques	fait
Plugin du serveur, protocole avec des volontaires, conteneurs, relevé de temps, vidéo	à faire

Le projet a avancé par étapes, chacune déclenchée par un problème vu en test. Début octobre, le bot décidait une fois toutes les 30 secondes et n'arrivait même pas à fabriquer une pioche en bois. Le 5 octobre, on a repris les actions de Mindcraft. Le 6, le code s'est mis à calculer la prochaine étape d'un objectif (bois, pioche, pierre, fer...), et le modèle n'a plus qu'à la choisir. Avec ces deux changements, le bot a fabriqué un casque en diamant au cours d'un test. Du 7 au 11 octobre, on a revu la façon dont le bot parle et agit, jusqu'à la solution décrite en partie 3. Le 21 octobre, on est passés d'un seul programme pour les trois bots à trois programmes séparés, une IA par bot. Le 22, on a lancé les premiers tests de coopération.

Pour que les trois bots ne se ressemblent pas, chacun a un profil de joueur. On est d'abord partis des types de joueurs de Bartle (1996), le classement le plus connu : un bot "chill" (le socializer, qui discute et aide), un "rusher" (l'achiever, qui veut s'équiper vite) et un "builder". Ce dernier vient de la critique de Bartle par Nick Yee (2006), où la motivation "immersion" comprend la construction. Le 21 octobre, on est passés au modèle de motivation de Quantic Foundry, du même Nick Yee, plus récent et mesuré sur de vrais joueurs (six grandes motivations : action, social, maîtrise, accomplissement, immersion, créativité). Joueur4 est devenu un fermier chaleureux (social et immersion), Joueur5 un mineur sec qui parle avec des chiffres (accomplissement et maîtrise), et Joueur6 une bâtisseuse posée et polie (créativité et immersion). Chaque bot a aussi une petite vie hors du jeu : tous sont des étudiants majeurs, de filières et de villes différentes, pour pouvoir parler d'autre chose que de Minecraft. Ils ont aussi un caractère (plus ou moins serviable, plus

ou moins flemmard) et une humeur tirée au hasard à chaque partie. C'est le modèle qui décide à partir de ces traits s'il aide quelqu'un tout de suite, plus tard, ou pas du tout.

Ces tests sont encourageants. Lors d'une partie d'une heure où les trois bots devaient construire ensemble une maison et une ferme (912 appels au modèle, aucune erreur), ils se sont réparti le travail et se répondaient vraiment. Joueur5 écrit "jsuis mort en 1 1. jai plus rien", et Joueur4 lui répond "oh non grave je te ramene un fer". Lors de tests de 10 minutes en mode créatif, chaque bot a inventé et construit une structure : une petite maison de 52 blocs, une grande maison de 78 blocs, une tour de guet. Ces plans sont gardés dans une bibliothèque, pour pouvoir les refaire. Dans un test de chaîne d'entraide (un bot fournit le bois, un autre va jusqu'au fer, le dernier jusqu'aux diamants), les bots se sont demandé et donné des objets dans le chat dès les premières minutes.

3. Point technique charnière : trois IA indépendantes qui parlent de façon logique

Le problème

Pour passer pour un humain, un bot doit répondre quand on lui parle, et ce qu'il dit doit correspondre à ce qu'il fait. Au départ, ce n'était pas le cas. Sur une partie d'une heure, 45 % des messages des bots racontaient ce qu'ils faisaient ("jfais des planches"), et seulement 23 % répondaient à un autre joueur. Certains messages restaient sans réponse : "joueur4 tas de la pierre ?" alors qu'il en avait 40.

Ce qu'on a essayé avant

D'abord, on a séparé une "bouche", qui répond au chat, et des "mains", qui jouent. Le bot répondait vite, mais sans lien avec ce qu'il faisait : il écrivait "ok jarrive" pendant que ses mains continuaient à miner. Le projet Project Sid (Altera, 2024, des centaines d'agents dans Minecraft) décrit exactement ce problème.

On a donc essayé l'inverse : un seul cerveau qui décide en même temps de l'action et de ce que le bot dit, comme Mindcraft. Le bot devenait cohérent, mais il ne répondait presque plus. La parole n'était qu'une option parmi une vingtaine d'actions et environ 4 000 tokens de description du jeu, et le modèle l'oubliait. On l'a vu grâce à un banc d'essai qu'on a écrit pour l'occasion : des messages de chat préparés à l'avance, passés au vrai code du bot avec le vrai modèle, sans lancer le jeu.

Un autre problème venait de l'organisation. Les trois bots tournaient dans un seul programme : ils partageaient une seule file d'appels au modèle, et un bot qui calculait un long chemin figeait les deux autres. Si le programme plantait, les trois bots tombaient en même temps.

La solution retenue

Aujourd'hui, chaque bot est un programme séparé, avec sa propre IA. Si un bot plante, il est relancé seul. Dans chaque programme, le bot a plusieurs "têtes" :

- des réflexes, écrits en code, pour se défendre ou fuir sans attendre le modèle ;
- la parole, un appel au modèle qui ne sert qu'à répondre dans le chat ;
- les actions, un autre appel, qui choisit quoi faire dans le jeu et ne parle jamais ;
- la réflexion, toutes les 3 minutes, qui résume la partie et donne un avis sur les autres joueurs.

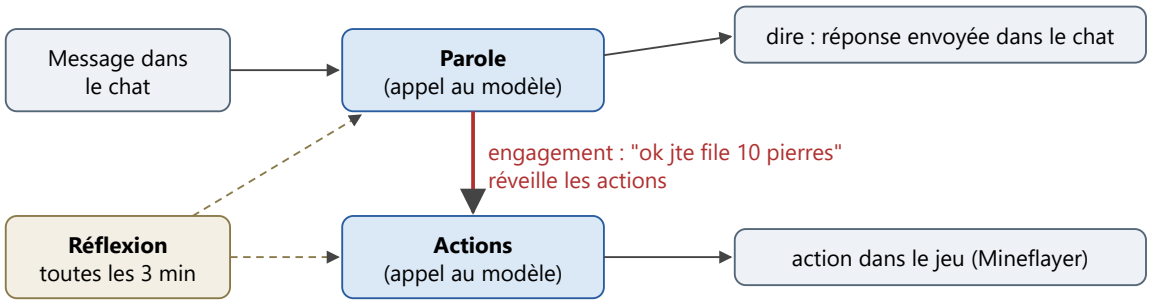


Figure 1. Les têtes d'un bot (un programme par bot) : la parole répond, et sa promesse (engagement) déclenche l'action.

Ce qui rend le bot logique, c'est le lien entre la parole et les actions. La réponse de la parole a une forme imposée, avec trois champs : à qui et à quoi il répond (`repond_a`), son message (`dire`), et ce qu'il promet

de faire tout de suite (`engagement`). Remplir `repond_a` en premier l'oblige à rester sur le sujet. Si le bot écrit "ok jte file 10 pierres", l'engagement réveille la tête des actions, qui reçoit "tu t'es engagé à donner 10 pierres" et le fait. Il ne peut donc plus dire une chose et en faire une autre.

La consigne de la parole est courte et centrée sur la conversation. Le fil de discussion lui est présenté comme un dialogue (qui parle à qui), et le message auquel il doit réagir est placé à la fin. Avant d'envoyer un message, le code vérifie aussi qu'il n'est pas trop long, qu'il ne répète pas un message précédent, qu'il ne contient pas d'insulte grave ni de nom technique ("oak_log" au lieu de "bois"). Sinon, le modèle doit reformuler.

Les résultats

Message reçu	Réponse du bot après le changement
"joueur4 tas de la pierre ?"	"jen ai une quarantaine si tu veux"
"tes un bot ou quoi"	"nan t'es sérieux"
"il pleut chez toi ?"	une vraie réponse sur la météo
"jai pas de bouffe"	"j'ai 3 steaks, prends si tu veux"

Pour mesurer plus largement, on a ajouté un juge automatique : le modèle note chaque réponse du bot, en donnant d'abord sa raison, puis son verdict. Sur 5 situations tirées d'une vraie partie, avec 3 essais chacune, 12 à 13 réponses sur 15 sont jugées correctes. En partie, les bots se répondent maintenant entre eux et se demandent des objets dans le chat.

4. Limites et échecs

La maison et la ferme du test d'une heure n'ont pas été construites. Le point d'apparition était tombé dans un marais, sans terrain plat ni pierre en surface, et un bot promettait des objets qu'il n'avait pas (37 dons ratés). Le code refuse maintenant de donner un objet absent de l'inventaire, ou de miner un bloc sans le bon outil. Dans la chaîne d'entraide, les rôles se sont mélangés : le bot des diamants a fabriqué lui-même les outils du bot du fer, parce que notre scénario ne lui interdisait pas la pierre. Il faut aussi mieux placer les structures : quand un bot ajoute un toit, il le pose là où il se trouve, et non sur sa maison.

Côté chat, les bots ont encore des tics de langage ("frero" dans 24 % des réponses au banc d'essai, alors que l'argot ne fait qu'environ 5 % des messages chez de vrais joueurs), et ils lancent rarement la conversation d'eux-mêmes. Le juge automatique se trompe parfois : il a noté "wsh frero" hors sujet, alors que c'est un salut. Enfin, la passerelle de l'université a été coupée du 19 au 22 octobre pour une mise à jour. On a vérifié que la connexion échouait aussi sans notre code, depuis la VM, nos PC et un service extérieur.

5. Coût

Avoir une IA par bot et une parole séparée demande plus d'appels au modèle. Avec trois bots, on compte environ 30 appels par minute, soit environ 4,2 millions de tokens pour une heure de jeu (mesuré le 22/10). Chaque bot a une limite de tokens dans le code. Côté temps, la partie charnière a demandé 4 séances du journal (7, 8, 9 et 11 octobre), plus une pour le passage à trois programmes (21 octobre). Le relevé de temps précis n'est pas encore en place. En complexité, on a ajouté un module de parole, un script qui lance un programme par bot, et un banc d'essai du chat, réutilisé à chaque modification.

6. Suite

On veut d'abord faire coopérer les bots jusqu'au bout, avec des tests courts et ciblés avant les parties longues, et remplacer les métiers fixes (mineur, bâtisseuse, fermier) par des préférences : tout le monde peut miner et construire, chacun le fait plus ou moins. Le plugin du serveur n'est pas commencé. Il servira à proposer des tâches à faire ensemble et à recueillir les désignations, et son fonctionnement reste à définir en groupe. Viendront ensuite le protocole avec des volontaires (consentement, données pseudonymisées),

les parties mixtes humains et bots, les conteneurs, la vidéo en anglais et la note finale, avant la soutenance du 27 novembre.